

AWS IAM Deep Dive

Users, Groups, Roles, MFA & RBAC Lab Guide

Step-by-Step Instructions from Scratch

What You Will Learn

- Create IAM users and organize them into groups
- Write and attach custom least-privilege policies
- Enable Multi-Factor Authentication (MFA) for users
- Create IAM roles and test role assumption (Assume Role)
- Verify authentication and authorization using RBAC principles

Estimated Time: 75 – 100 minutes

Skill Level: Beginner to Intermediate

Overview & Concepts

Key IAM Terms

Term	Definition
User	A single identity (person or application) with long-term credentials.
Group	A collection of users that share the same permissions via attached policies.
Role	A temporary identity with its own permissions, assumed by users, services, or accounts — no long-term credentials.
Policy	A JSON document that defines what actions are allowed or denied on which resources.
MFA	Multi-Factor Authentication — a second proof of identity (e.g., a one-time code) in addition to a password.
Least Privilege	Granting only the exact permissions required to perform a task — nothing more.
RBAC	Role-Based Access Control — permissions are attached to roles/groups, and identities inherit access by belonging to them.

Resource Reference Table

Use this table as a quick reference throughout the lab:

Resource	Name	Purpose
Group	developers-group	Read-only S3 access for the development team
Group	s3-admins-group	Full S3 access for storage administrators
User	alice-dev	Member of the developers group, MFA-enabled
User	bob-admin	Member of s3-admins-group, MFA enabled
Custom Policy	S3ReadOnlyCustomPolicy	Least-privilege read-only access to one S3 bucket
Custom Policy	S3FullAccessCustomPolicy	Full access to one S3 bucket, scoped by ARN
Role	S3ReadOnlyRole	Assumable role granting temporary read-only S3 access
Trust Policy	(embedded in role)	Defines who is allowed to assume the S3ReadOnlyRole

♦ **Note:** Replace bucket names, account IDs, and ARNs shown in this guide with your own values as you go.

Task 1 — Create IAM Users and Groups

Groups allow you to manage permissions for multiple users simultaneously. In this task, you'll create two groups representing different job functions, then create users and add them to the appropriate group.

Step 1.1 Create an S3 Bucket for Testing

1. Sign in to the AWS Console as an administrator and search for S3 in the console search bar.
2. Click Create bucket.
3. Enter a globally unique bucket name, e.g., **iam-lab-bucket-yourname-2026**.
4. Leave all other settings at their defaults and click Create bucket.
5. Upload a small test file (e.g., test.txt) into the bucket so you have something to verify access against later.

💡 **Tip:** note down the bucket ARN (arn: aws: s3: your-bucket-name) — you'll need it when writing custom policies in Task 2.

Step 1.2 Create the developers group

6. In the console search bar, type IAM and open the IAM service.
7. In the left panel, click User groups → Create group.
8. Enter the group name: **developers-group**.
9. Under Attach permissions policies, search for and check **AmazonS3ReadOnlyAccess** (a temporary AWS-managed policy — you'll replace it with a custom least-privilege policy in Task 2).
10. Click Create group.

Step 1.3 Create the s3-admins-group

11. Click User groups → Create group again.
12. Enter the group name: **s3-admins-group**.
13. Under Attach permissions policies, search for and check **AmazonS3FullAccess** (temporary — replaced with a custom policy in Task 2).
14. Click Create group.

✅ **Groups ready!** You now have two groups representing different levels of access — this is the foundation of RBAC.

Step 1.4 Create the alice-dev User

15. In the left panel, click Users → Create user.
16. Enter User name: **alice-dev**.

17. Check: Provide user access to the AWS Management Console (so we can test console sign-in later).
18. Choose I want to create an IAM user and **set a custom password; enter a temporary password** and require a **password reset at next sign-in**.
19. Click Next. Under Permissions options, select **Add user to group** and check the **developers-group**.
20. Click Next, review, and click Create user.
21. Download the **.csv with the sign-in URL, username, and password**, or **copy them somewhere safe** — this is shown only once.

Step 1.5 Create the **bob-admin** User

22. Click Users → Create user again.
23. Enter User name: **bob-admin**, enable console access, and set a temporary password with a required reset.
24. Under Permissions options, select Add user to group and **check s3-admins-group**.
25. Click Next, review, and click Create user.
26. Save the sign-in credentials for bob-admin.

✅ **Users created!** alice-dev inherits developers-group permissions, and bob-admin inherits s3-admins-group permissions — neither user has policies attached directly.

Task 2 — Write and Attach Custom Least-Privilege Policies

AWS-managed policies like `AmazonS3ReadOnlyAccess` are convenient but often broader than necessary — they apply to every S3 bucket in the account. In this task, you'll replace them with custom policies scoped to only your test bucket.

Step 2.1 Create the Read-Only Custom Policy

27. In the IAM console left panel, click Policies → Create policy.
28. Click the JSON tab and replace the default content with the following, substituting your bucket name:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucketOnly",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::iam-lab-bucket-yourname-2026"
    },
    {
      "Sid": "ReadObjectsOnly",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::iam-lab-bucket-yourname-2026/*"
    }
  ]
}
```

29. Click Next. Name the policy **S3ReadOnlyCustomPolicy** and add a description: **"Least-privilege read-only access to the IAM lab bucket."**
30. Click Create policy.

💡 **Why this is least privilege:** it grants only `s3: ListBucket` and `s3: GetObject` on one specific bucket ARN — not `s3: *` on every bucket in the account, like the AWS-managed policy does.

Step 2.2 Create the Full-Access Custom Policy

31. Click Policies → Create policy → JSON tab, and enter:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "FullAccessToLabBucket",
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::iam-lab-bucket-yourname-2026",
        "arn:aws:s3:::iam-lab-bucket-yourname-2026/*"
      ]
    }
  ]
}
```

32. Click Next. Name the policy **S3FullAccessCustomPolicy**.
33. Click Create policy.

Step 2.3 Swap the Managed Policies for Custom Policies

34. Go to **User groups** → **developers-group** → **Permissions tab**.
35. Select **AmazonS3ReadOnlyAccess** and click Remove.
36. Click **Add permissions** → **Attach policies**, search for **S3ReadOnlyCustomPolicy**, check it, and click Add permissions.
37. Repeat the same swap for **s3-admins-group**: **remove AmazonS3FullAccess** and attach **S3FullAccessCustomPolicy**.

✓ **Least privilege applied!** Each group now grants access to exactly one bucket instead of every bucket in the account.

Task 3 — Configure Multi-Factor Authentication (MFA)

MFA adds a second layer of proof — something you have (a code from an authenticator app) in addition to something you know (your password). You'll enable it for both test users.

Step 3.1 Install an Authenticator App

38. On your phone or computer, install an authenticator app such as Google Authenticator, Microsoft Authenticator, or Authy.

Step 3.2 Enable MFA for alice-dev

39. Sign out of the administrator account, or open a private/incognito browser window.
40. Sign in to the AWS Console using the account ID/alias, username `alice-dev`, and the temporary password from Step 1.4. Set a new permanent password when prompted.
41. Click the account name in the top-right corner and select Security credentials.
42. Under multi-factor authentication (MFA), click Assign MFA device.
43. Choose the Authenticator app and click Next.
44. Scan the displayed QR code with your authenticator app (or enter the secret key manually).
45. Enter two consecutive MFA codes from the app, one after the other, into the MFA code 1 and MFA code 2 fields.
46. Click Add MFA. The console confirms the device is now assigned.

✔ **MFA enabled!** `alice-dev` must now supply a one-time code on every sign-in, in addition to her password.

Step 3.3 Enable MFA for bob-admin

47. Repeat Step 3.2 for `bob-admin`: sign in with his temporary credentials, set a permanent password, and assign an MFA device under Security credentials.

💡 **Tip:** you can add a second virtual MFA device for the same user by repeating this process — useful if you also want to test recovery scenarios.

Step 3.4 Enforce MFA with a Policy (Optional)

⚠ **Careful:** test this policy with a non-critical account first. If applied incorrectly, users without MFA already configured could lock themselves out of most actions.

48. As the administrator, create a new customer-managed policy named **RequireMFA** using the JSON below — it denies almost all actions unless the request was authenticated with MFA:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowViewAccountInfo",
      "Effect": "Allow",
      "Action": [
        "iam:GetAccountSummary",
        "iam:ListVirtualMFADevices"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyAllExceptMFAWhenNotMFA",
      "Effect": "Deny",
      "NotAction": [
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ListMFADevices",
        "iam:ResyncMFADevice",
        "sts:GetSessionToken"
      ],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }
      }
    }
  ]
}
```

49. Attach RequireMFA to both the **developers-group** and **s3-admins-group**.

Task 4 — Create IAM Roles and Test Role Assumption

Roles provide temporary credentials instead of long-term access keys. In this task, **alice-dev**, who normally only has **read-only access**, will assume a role that temporarily grants broader access, demonstrating how RBAC and role assumption work together.

Step 4.1 Create the S3ReadOnlyRole

50. Sign back in as the administrator.
51. In the IAM console, click Roles → Create role.
52. Under Trusted entity type, select AWS account.
53. Choose This account, so any authorized identity in your account can attempt to assume the role.
54. Click Next. Search for and attach S3ReadOnlyCustomPolicy (created in Task 2).
55. Click Next, name the role S3ReadOnlyRole, and add a description: "Assumable role for temporary S3 read-only access."
56. Click Create role.

Step 4.2 Restrict Who Can Assume the Role (Trust Policy)

57. Open S3ReadOnlyRole and click the Trust relationships tab.
58. Click Edit trust policy and replace the content with the following, substituting your account ID:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<YOUR_ACCOUNT_ID>:user/alice-dev"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "Bool": { "aws:MultiFactorAuthPresent": "true" }
      }
    }
  ]
}
```

59. Click Update policy.

🔴 **Why the MFA condition matters:** this trust policy only allows the role to be assumed if the calling identity's session was authenticated with MFA — combining Task 3 and Task 4.

Step 4.3 Allow alice-dev to assume the Role

60. Create a new customer-managed policy named AssumeS3ReadOnlyRole with the following JSON, substituting your account ID:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::<YOUR_ACCOUNT_ID>:role/S3ReadOnlyRole"
    }
  ]
}
```

61. Attach AssumeS3ReadOnlyRole directly to the alice-dev user (or create a small group for it, e.g., role-assumers-group, and add alice-dev to it).

Step 4.4 Test Role Assumption in the Console

62. Sign in as alice-dev (completing the MFA prompt).
63. Click the account name in the top-right corner and select Switch role.
64. Enter your Account ID (or alias) and Role name: S3ReadOnlyRole. Optionally set a display name/color.
65. Click Switch Role.
66. The top-right banner now shows you are operating as S3ReadOnlyRole instead of alice-dev.

✅ **Role assumed!** alice-dev is temporarily using the permissions of S3ReadOnlyRole. These credentials automatically expire, unlike a permanent access key.

Step 4.5 Test Role Assumption via the CLI (Optional)

67. On a machine with the AWS CLI configured for alice-dev's access key, run:

```
aws sts assume-role \
  --role-arn arn:aws:iam::<YOUR_ACCOUNT_ID>:role/S3ReadOnlyRole \
```

```
--role-session-name alice-session \  
--serial-number arn:aws:iam::<YOUR_ACCOUNT_ID>:mfa/alice-dev \  
--token-code <CURRENT_MFA_CODE>
```

68. The command returns temporary `AccessKeyId`, `SecretAccessKey`, and `SessionToken` values, valid for a limited time (default 1 hour).
69. Export them as environment variables and run an S3 command to confirm the temporary credentials work:

```
export AWS_ACCESS_KEY_ID=<TEMP_ACCESS_KEY>  
export AWS_SECRET_ACCESS_KEY=<TEMP_SECRET_KEY>  
export AWS_SESSION_TOKEN=<TEMP_SESSION_TOKEN>  
aws s3 ls s3://iam-lab-bucket-yourname-2026
```

 **Tip:** the `--serial-number` and `--token-code` flags are required because the trust policy demands MFA. Without them, `AssumeRole` is denied.

Task 5 — Verify Authentication and Authorization Using RBAC

In this final task, you will deliberately test both successful and denied actions to confirm that permissions flow correctly through groups and roles — the core idea behind Role-Based Access Control.

Step 5.1 Verify Alice-Dev's Baseline (Read-Only) Access

70. Sign in as alice-dev (not switched to any role) and complete the MFA prompt.
71. Open the S3 console and navigate into iam-lab-bucket-yourname-2026.
72. Confirm you can view and download test.txt (allowed by S3ReadOnlyCustomPolicy via developers-group).
73. Attempt to upload a new file into the bucket.

 **Expected result:** the upload should fail with an Access Denied error, since the developers group only grants s3:ListBucket and s3:GetObject — not s3:PutObject.

Step 5.2 Verify Bob-Admin's Full Access

74. Sign in as bob-admin and complete the MFA prompt.
75. Open the S3 console and navigate to the same bucket.
76. Upload a new test file, then delete it.

 **Expected result:** both actions succeed, since s3-admins-group grants s3:* on the bucket via S3FullAccessCustomPolicy.

Step 5.3 Verify Role Assumption Changes Permissions

77. While signed in as alice-dev, switch to S3ReadOnlyRole as in Step 4.4.
78. Confirm you can still list and download objects (same permissions as her group, in this case).
79. Switch back to alice-dev (click Back to alice-dev in the top-right banner).

Step 5.4 Verify MFA Enforcement

80. Sign out completely, then attempt to sign in as alice-dev again.
81. After entering the correct password, confirm the console prompts for an MFA code before granting access.
82. Intentionally enter an incorrect MFA code once to confirm the sign-in is rejected, then enter the correct code to complete sign-in.

 **Expected result:** sign-in fails with the wrong code and succeeds only with a valid one, confirming MFA is actively enforced.

Step 5.5 Verify the Trust Policy Blocks Unauthorized Role Assumption

83. Sign in as bob-admin and attempt Switch Role using S3ReadOnlyRole and your account ID.

 **Expected result: the switch should fail, since the trust policy in Step 4.2 names only alice-dev as an authorized principal.**

Step 5.6 RBAC Verification Summary

Test	Identity Used	Expected Outcome
Read object from bucket	alice-dev	Allowed
Upload object to the bucket	alice-dev	Denied
Upload & delete object	bob-admin	Allowed
Sign in without MFA code	alice-dev / bob-admin	Denied (blocked at MFA prompt)
Assume S3ReadOnlyRole	alice-dev	Allowed
Assume S3ReadOnlyRole	bob-admin	Denied (not in trust policy)

 **Lab complete!** You've confirmed that authentication (who you are, enforced by password + MFA) and authorization (what you can do, enforced by group and role policies) are working together correctly.

Cleanup (Recommended After Testing)

- Delete the test users: IAM console → Users → select alice-dev and bob-admin → Delete.
- Delete the groups: IAM console → User groups → select developers-group and s3-admins-group → Delete.
- Delete the role: IAM console → Roles → select S3ReadOnlyRole → Delete.
- Delete the custom policies: IAM console → Policies → select each custom policy → Delete (only possible after they are detached from all identities).
- Empty and delete the S3 test bucket: S3 console → select bucket → Empty, then Delete bucket.

⚠ Reminder: IAM resources are free, but keeping unused test users, roles, and access keys around is a security risk. Clean them up once you're done experimenting.